ISYE 7406 HW3

Introduction

Local smoothing is a way to smooth out noisy data without assuming a fixed pattern. Instead of fitting a single equation to the entire dataset, local smoothing looks at small sections of data at a time. This makes it useful for finding trends that change over different parts of the dataset. In this analysis, three smoothing methods are explored: Loess smoothing, Nadaraya-Watson kernel smoothing, and Spline smoothing. These methods are compared by utilizing two datasets: an equidistant and an non-equidistant dataset. Simple exploratory data analysis is conducted on the datasets to address the nature of the observations.

The objective is to investigate how these smoothing models handle different datasets by analyzing their mean squared error in addition to identifying their performances in terms of bias and variance separately.

Exploratory Data Analysis

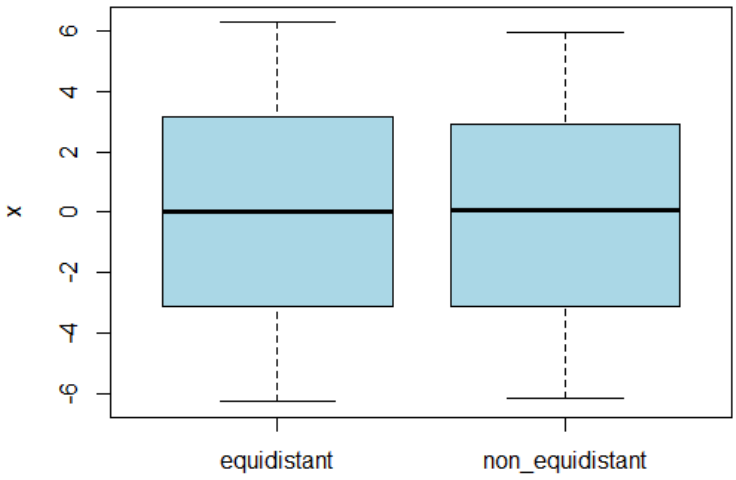

Figure 1: Boxplot of equidistant and non-equidistant dataset

The first figure illustrates a boxplot of the equidistant and the non-equidistant observations. The formula to generate a dataset of the form (x_i, Y_i) is as follows:

$f(x) = 2\pi(-1 + 2\frac{i-1}{n-1})$

Based on this, it is expected for the equidistant boxplot to be distributed evenly. Conversely, the non-equidistant typically uses a formula that includes random number generation, but for the

purposes of this assignment it uses a dataset with predetermined values while still following the formula's constraints. Nonetheless, it still shows uneven distribution of the dataset as expected.

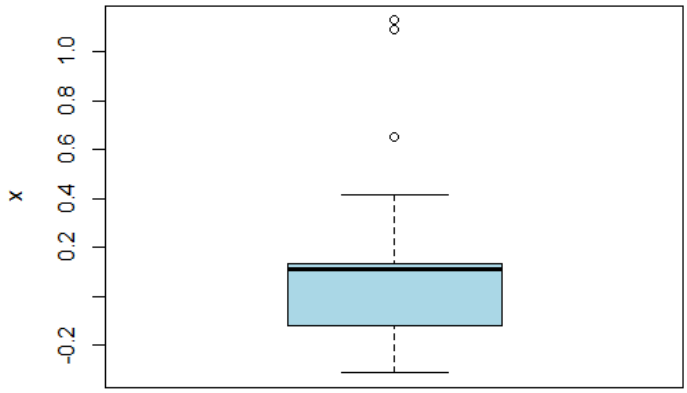

Figure 2: Boxplot of Y values

The second figure is a boxplot showcasing the distribution of the noisy observed values in the dataset. These values are generated based on the Mexican-hat function and adds a Gaussian noise of mean = 0 and sd = 0.2. The majority of Y values are concentrated around 0, and the whiskers are relatively symmetrical. There are some significant outliers, which is bolstered by the squaring effect. This implies that Loess may over-smooth (making the curve much smoother) and struggle to capture important details, and Nadaraya-Watson may struggle with its sensitivity to noise, resulting in increased variance. Spline smoothing, in contrast, is likely to strike a better balance by adapting to local variations while mitigating excessive noise.

Methods

Using Monte Carlo 1000 runs, 3 smoothing techniques are applied onto the equidistant dataset:

- Local Regression (Loess): Many small polynomials in different sections of a data using a span = 0.75
- Nadaraya-Watson Kernel Smoothing (NW): Obtains the weights averages of nearby points and applies a Gaussian kernel that emphasizes more on points closer to the kernel, and uses a bandwidth of 0.2.
- Spline Smoothing: Splitting up data into small sections with a fitted polynomial for each with a default tuning parameter. Unlike Loess, Spline emphasizes a smoother transition between each of the sections.

Each method is compared by plotting the fitted means, bias, variance, and mean squared error. Finally, the same steps are repeated for the non-equidistant dataset but with changes to certain parameters for a more meaningful comparison:

- Span = 0.3365
- Spar = 0.7163

Results

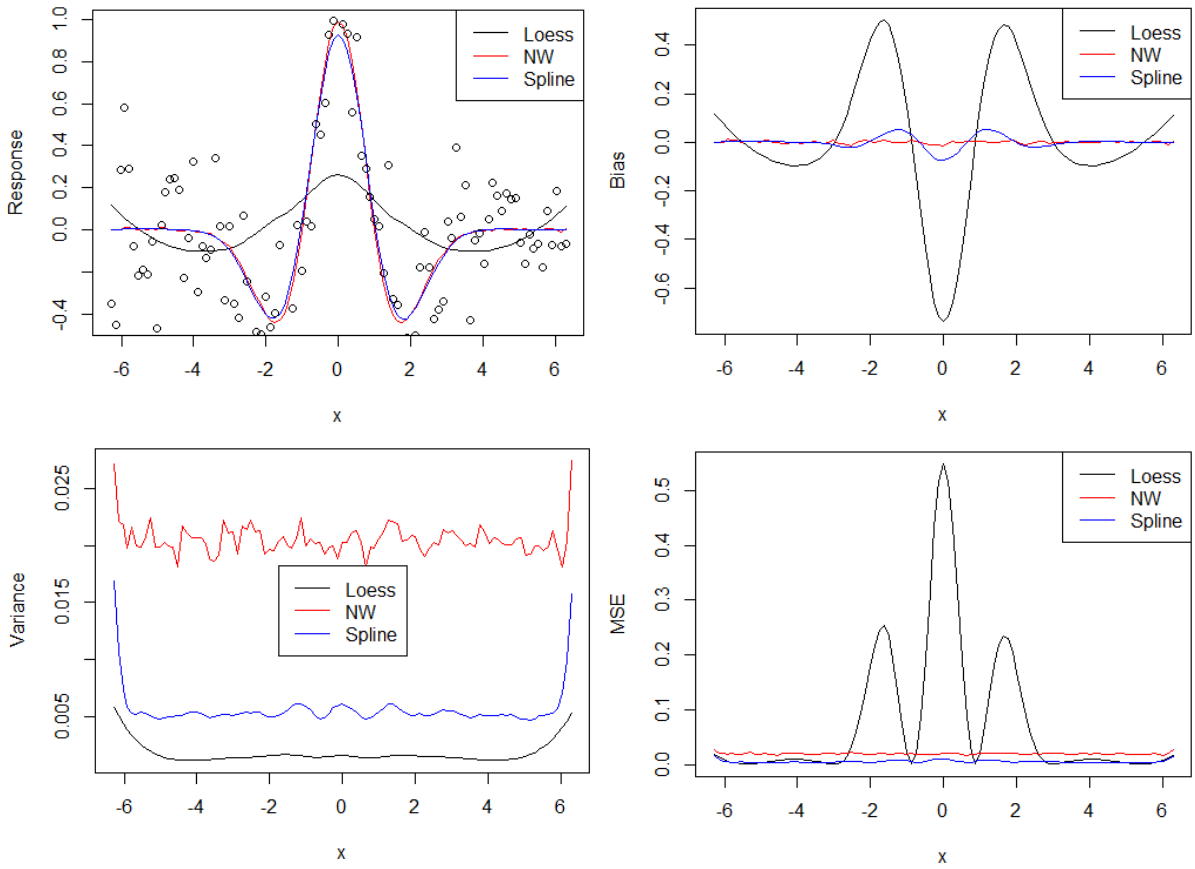

Figure 3 (clockwise starting from top left): Fitted Plot, Bias, MSE, Variance plots for equidistant points

Beginning with the fitted plots from Figure 3, Loess has a smoother polynomial than NW and Spline. However, the raw observations (scatter points) seem to fit NW and Spline much better, as the scatter points reflect significantly greater peak and troughs than the Loess pattern. This is further evidenced by the bias plot, which indicates that Loess severely overestimates and underestimates from the true function. NW has the upper hand in the bias plot due to its general flatness along the axis, and Spline is only a bit more turbulent. Variance tells the opposite, as Loess showcases the lowest sensitivity to fluctuations in the dataset, indicating the best accuracy among the three smoothing methods. Spline has the middle variance, and NW has the greatest fluctuations of varying sample data. The tradeoff is illustrated by the contrasting relationship between the bias and variance of the three models: underfitting for high bias and low variance (Loess) and overfitting for low bias and high variance (NW). Therefore, MSE aims to strike a balance between these two factors. According to the MSE plot, Loess has the highest error especially when near sharp peaks with rapid changes. NW and Spline are much closer in MSE, and Spline is slightly better with the lowest MSE across all the equidistant data.

It is worth considering that the parameters were predetermined for LOESS and NW, but not for Spline, which can raise concerns for fairness. It would have been beneficial to conduct cross-validation to optimize the parameters for all methods.

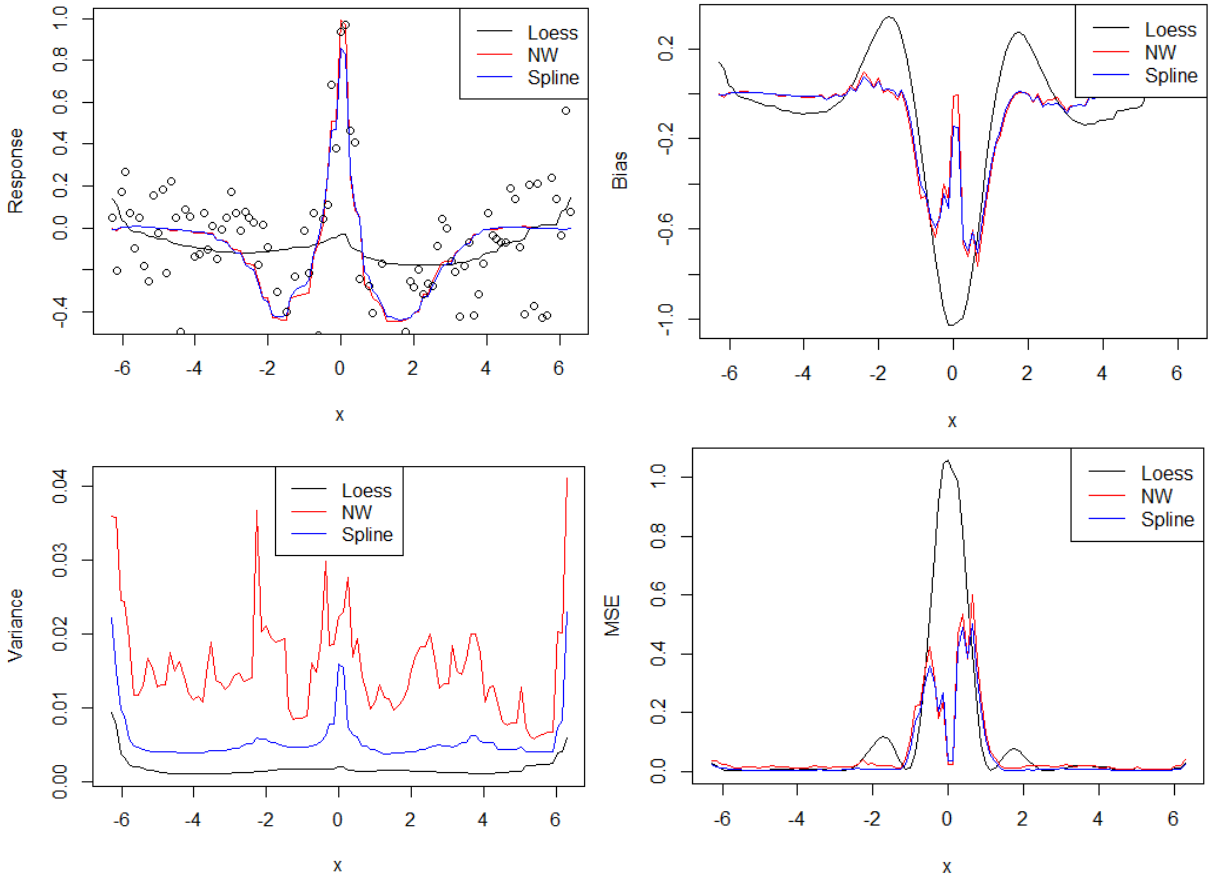

Figure 4 (clockwise starting from top left): Fitted Plot, Bias, MSE, Variance plots for non-equidistant points

Results from Figure 4 show the fitted plot, bias, variance, and MSE plots for the non-equidistant points. Once again, Loess performs the worst on the fitted plot with the exception of struggling with noise and extreme swings near x=0, and NW and Spline are performing similarly well in the fitted plot. A notable difference in the bias plot this time around is that NW and Spline have a higher bias in the non-equidistant dataset than the equidistant counterpart, with significant fluctuations happening near the center. Loess' bias performance is extremely similar to the previous case and has the greatest bias. Variance order remains the same, but with a lot more variance across all methods. Interestingly, Spline remains much more effective at smoothing out variances, whereas NW appears to suffer from more turbulent variance changes. Finally, the MSE plot shows roughly equal performances in the outer points, but every method has a significant proportional increase as x approaches 0. While not by much, it would appear that Spline has the lowest MSE. Ultimately, it would appear that Spline has the least MSE and bias, Loess has the lowest variance but the greatest bias and difference in fitted plot, and NW has the greatest variance, but largely falls in the middle for the other plots.

In a non-equidistant set of points, there is an uneven application of smoothing methods. It is evident that Loess and NW would struggle more because of their reliance on a nicely distributed local area (Loess in particular). Parameters are also a concern because the span may be better suited for certain areas, and not ideal for the others. Generally speaking, Spline is inherently advantageous due to its adaptability to non-uniform patterns.

Findings

In both equidistance and non-equidistant datasets, Loess performs the worse, as it struggles with capturing sharp peaks and showing the highest bias and MSE. NW suffers from high variance and low bias in both cases, which strongly suggests overfitting. NW performs closest to Spline, but is ultimately not the top performer in MSE due to its high sensitivity to sample fluctuations. Spline is by far the best performer, having the least MSE in both cases, as well as having the least bias in the non-equidistant dataset.

Consider that Loess and NW had predetermined parameters, while a spar parameter was not set for Spline in the equidistant set of points. An improvement would include optimizing parameters through cross-validation techniques, which could have helped to reduce bias or variance.

Appendix

```
## Part #1 deterministic equidistant design
## Generate n=101 equidistant points in [-2\pi, 2\pi]
m <- 1000
n <- 101
x <- 2*pi*seq(-1, 1, length=n)
## Initialize the matrix of fitted values for three methods
fvlp <- fvnw <- fvss <- matrix(0, nrow= n, ncol= m)
##Generate data, fit the data and store the fitted values
for (j in 1:m){
  ## simulate y-values
  ## Note that you need to replace $f(x)$ below by the mathematical definition
in eq. (2)
  y <- (1-x^2)*exp(-0.5*x^2) + rnorm(length(x), sd=0.2);
  ## Get the estimates and store them
  fvlp[,j] <- predict(loess(y ~ x, span = 0.75), newdata = x);
  fvnw[,j] <- ksmooth(x, y, kernel="normal", bandwidth= 0.2, x.points=x)$y;
  fvss[,j] <- predict(smooth.spline(y ~ x), x=x)$y
}
## Below is the sample R code to plot the mean of three estimators in a single
plot
meanlp = apply(fvlp,1,mean);
meannw = apply(fvnw,1,mean);
meanss = apply(fvss,1,mean);
dmin = min( meanlp, meannw, meanss);
dmax = max( meanlp, meannw, meanss);
matplot(x, meanlp, "l", ylim=c(dmin, dmax), ylab="Response")
matlines(x, meannw, col="red")
matlines(x, meanss, col="blue")
legend("topright", legend=c("Loess", "NW", "Spline"), col=c("black", "red",
"blue"), lty=1)
## You might add the raw observations to compare with the fitted curves
points(x,y)
## Can you adapt the above codes to plot the empirical bias/variance/MSE?
mex_hat <- (1-x^2)*exp(-0.5*x^2)

# Bias plot
biaslp <- meanlp - mex_hat
biasnw <- meannw - mex_hat
biasss <- meanss - mex_hat
dmin_2 <- min(biaslp, biasnw, biasss)
dmax_2 <- max(biaslp, biasnw, biasss)
matplot(x, biaslp, "l", ylim=c(dmin_2, dmax_2), ylab="Bias")
matlines(x, biasnw, col="red")
matlines(x, biasss, col="blue")
legend("topright", legend=c("Loess", "NW", "Spline"), col=c("black", "red",
"blue"), lty=1)
```

```r
# Variance plot
varlp <- rowMeans((fvlp - meanlp)^2)
varnw <- rowMeans((fvnw - meannw)^2)
varss <- rowMeans((fvss - meanss)^2)
dmin_3 <- min(varlp, varnw, varss)
dmax_3 <- max(varlp, varnw, varss)
matplot(x, varlp, "l", ylim=c(dmin_3, dmax_3), ylab="Variance")
matlines(x, varnw, col="red")
matlines(x, varss, col="blue")
legend("center", legend=c("Loess", "NW", "Spline"), col=c("black", "red",
"blue"), lty=1)

# MSE plot
mselp <- rowMeans((fvlp - mex_hat)^2)
msenw <- rowMeans((fvnw - mex_hat)^2)
msess <- rowMeans((fvss - mex_hat)^2)
dmin_4 <- min(mselp, msenw, msess)
dmax_4 <- max(mselp, msenw, msess)
matplot(x, mselp, "l", ylim=c(dmin_4, dmax_4), ylab="MSE")
matlines(x, msenw, col="red")
matlines(x, msess, col="blue")
legend("topright", legend=c("Loess", "NW", "Spline"), col=c("black", "red",
"blue"), lty=1)

## Part #2 non-equidistant design
## assume you save the file "HW04part2-1.x.csv" in the local folder "C:/temp",
x2 <- read.table(file= "HW04part2-1.x.csv", header=TRUE);
x2_num <- as.numeric(x2[,1])
## within each loop, you can consider the three local smoothing methods:
## please remember that you need to first simulate Y values within each loop
fvlp <- fvnw <- fvss <- matrix(0, nrow= n, ncol= m)
for (j in 1:m){
  ## simulate y-values
  ## Note that you need to replace $f(x)$ below by the mathematical definition
in eq. (2)
  y <- (1-x2_num^2)*exp(-0.5*x2_num^2) + rnorm(length(x2_num), sd=0.2);
  ## Get the estimates and store them
  fvlp[,j] <- predict(loess(y ~ x2_num, span = 0.75), newdata = x2_num);
  fvnw[,j] <- ksmooth(x2_num, y, kernel="normal", bandwidth= 0.2,
x.points=x2_num)$y;
  fvss[,j] <- predict(smooth.spline(y ~ x2_num), x=x2_num)$y
}
predict(loess(y ~ x2, span = 0.3365), newdata = x2);
ksmooth(x2, y, kernel="normal", bandwidth= 0.2, x.points=x2)$y;
predict(smooth.spline(y ~ x2, spar= 0.7163), x=x2)$y

## Below is the sample R code to plot the mean of three estimators in a single
plot
```

```r
meanlp = apply(fvlp,1,mean);
meannw = apply(fvnw,1,mean);
meanss = apply(fvss,1,mean);
dmin = min( meanlp, meannw, meanss);
dmax = max( meanlp, meannw, meanss);
matplot(x, meanlp, "l", ylim=c(dmin, dmax), ylab="Response")
matlines(x, meannw, col="red")
matlines(x, meanss, col="blue")
legend("topright", legend=c("Loess", "NW", "Spline"), col=c("black", "red",
"blue"), lty=1)
## You might add the raw observations to compare with the fitted curves
points(x,y)
## Can you adapt the above codes to plot the empirical bias/variance/MSE?
mex_hat <- (1-x^2)*exp(-0.5*x^2)

# Bias plot
biaslp <- meanlp - mex_hat
biasnw <- meannw - mex_hat
biasss <- meanss - mex_hat
dmin_2 <- min(biaslp, biasnw, biasss)
dmax_2 <- max(biaslp, biasnw, biasss)
matplot(x, biaslp, "l", ylim=c(dmin_2, dmax_2), ylab="Bias")
matlines(x, biasnw, col="red")
matlines(x, biasss, col="blue")
legend("topright", legend=c("Loess", "NW", "Spline"), col=c("black", "red",
"blue"), lty=1)

# Variance plot
varlp <- rowMeans((fvlp - meanlp)^2)
varnw <- rowMeans((fvnw - meannw)^2)
varss <- rowMeans((fvss - meanss)^2)
dmin_3 <- min(varlp, varnw, varss)
dmax_3 <- max(varlp, varnw, varss)
matplot(x, varlp, "l", ylim=c(dmin_3, dmax_3), ylab="Variance")
matlines(x, varnw, col="red")
matlines(x, varss, col="blue")
legend("top", legend=c("Loess", "NW", "Spline"), col=c("black", "red", "blue"),
lty=1)

# MSE plot
mselp <- rowMeans((fvlp - mex_hat)^2)
msenw <- rowMeans((fvnw - mex_hat)^2)
msess <- rowMeans((fvss - mex_hat)^2)
dmin_4 <- min(mselp, msenw, msess)
dmax_4 <- max(mselp, msenw, msess)
matplot(x, mselp, "l", ylim=c(dmin_4, dmax_4), ylab="MSE")
matlines(x, msenw, col="red")
matlines(x, msess, col="blue")
```

```r
legend("topright", legend=c("Loess", "NW", "Spline"), col=c("black", "red",
"blue"), lty=1)


# EDA
x2_num <- as.numeric(x2[[1]])
y_num <- as.numeric(y[[1]])

hist(x2_num, breaks=10, main="Histogram of Equidistant Dataset", xlab="x",
col="lightblue", border="black")
combined_data <- data.frame(equidistant = x, non_equidistant = x2_num)
boxplot(combined_data, col="lightblue", ylab="x")
boxplot(y, main="Boxplot of Y values", col="lightblue", ylab="x")
hist(y_num, breaks=20, main="Histogram of Equidistant Dataset", xlab="x",
col="lightblue", border="black")
```